READVAULT

Personal Digital Library & PDF Reading App

Project Plan, Product Requirements, Architecture & Development Roadmap

Initial build: local-first • Budget: ■0 • Future-ready cloud migration

Prepared as a practical implementation guide

1. Executive Summary

ReadVault is a personal digital library designed to let one user upload, organize, read, and track PDF books from a clean web interface. The first release should run locally on a Windows development machine, use SQLite for metadata, and store PDF files on the local filesystem. Cloud accounts, multi-device synchronization, and advanced reading features should be added only after the local MVP is stable.

The product should be built in small, testable phases. Avoid premature microservices, paid infrastructure, and unnecessary dependencies. Keep the application architecture independent of the initial database and file-storage choices so that a later migration to Supabase is manageable.

2. Product Goals and MVP Scope

Primary goals

- Upload PDF files and maintain a searchable personal library.
- Open PDFs inside ReadVault rather than downloading them every time.
- Remember the last page read and resume from that position.
- Organize books using categories and, later, collections and tags.
- Provide a responsive experience on desktop and mobile browsers.

MVP priorities

1. Upload, list, edit, and delete PDF books.
2. Read PDFs in an embedded reader.
3. Save and restore reading progress automatically.
4. Search, filter, and sort the library.
5. Keep data and files locally until cloud migration is deliberately started.

Out of scope for the first MVP

- Multi-user SaaS, public sharing, and social features.
- AI summaries, AI chat with books, and text-to-speech.
- EPUB and other ebook formats.
- Cloud synchronization and mobile native applications.
- Complex analytics or recommendation engines.

3. Recommended Technology Stack

Layer	Initial choice	Later option
Application	Next.js App Router + TypeScript	Keep Next.js
UI	Tailwind CSS; shadcn/ui where useful	Keep
Database	SQLite (local)	Supabase PostgreSQL
PDF storage	Local filesystem	Private Supabase Storage
PDF reader	PDF.js	Keep
Validation	Zod	Keep

Layer	Initial choice	Later option
Forms	React Hook Form when needed	Keep
Version control	Git + GitHub	Keep
Deployment	Local development first	Vercel or persistent host after migration

Why local-first?

SQLite and local file storage avoid recurring costs during development. They are appropriate for a single-user local prototype. They are not a production cloud-storage strategy: serverless deployments generally do not provide durable local disk storage, and local files do not synchronize across devices.

4. System Architecture

Use a layered architecture so the user interface does not depend directly on SQLite or a particular storage provider.

- **Presentation layer:** Next.js pages, React components, layouts, and reader interface.
- **Application layer:** use cases for books, uploads, reading progress, bookmarks, collections, and statistics.
- **Repository layer:** interfaces for querying and updating book metadata and reading records.
- **Storage layer:** local PDF storage initially; Supabase Storage later.
- **Database layer:** SQLite initially; PostgreSQL after cloud migration.

Core rule: components should call application services or typed data-access functions, not issue ad hoc SQL or manipulate file paths directly.

5. Main Product Modules

Dashboard

- Total books, currently reading, completed, and progress overview.
- Continue-reading section and recently opened books.
- Quick upload, search, and navigation.

Library management

- Upload/create, view details, edit metadata, and delete.
- Grid/list layouts, search, category filters, and sorting.
- Track reading status and show book covers where available.
- Validate file type and size; detect duplicates using a file hash when implemented.

PDF reader

- PDF.js-based rendering with previous/next page and page jump.
- Zoom, fit width/page, fullscreen, and responsive controls.
- Keyboard shortcuts, text search, and PDF outline where supported.
- Mobile-friendly reading controls and optional dark reading theme.

Reading progress

- Automatically record the current page with debounced updates.
- Resume from the saved page when reopening a book.

- Store last-read time and reading status.
- Later: reading sessions, cross-device synchronization, and conflict handling.

Bookmarks and notes

- Bookmark a page and return to it later.
- Attach notes to a book or page.
- Later: highlights and searchable annotations.

Categories and collections

- Use categories for a primary classification.
- Use collections to group books across categories.
- Later: tags and multiple filters.

Statistics and goals

- Track completed books and pages read.
- Later: daily/weekly goals, streaks, and reading history.

Settings

- Reader preferences such as theme and default zoom.
- Later: account, storage usage, export, and backup settings.

6. Data Model

The local SQLite schema can use integer primary keys. When migrating to Supabase, map those IDs to UUIDs and preserve relationships. Keep user_id in the future-oriented schema even if the local MVP has only one user.

Suggested tables

- **profiles:** user profile and preferences.
- **books:** title, author, description, category, file metadata, page count, status, timestamps.
- **categories:** book classification.
- **collections:** named user-created groups.
- **collection_books:** many-to-many relationship between collections and books.
- **reading_progress:** current page, percentage, last-read timestamp; unique per user and book.
- **bookmarks:** saved page locations and optional labels.
- **notes:** notes associated with a book and optionally a page.
- **reading_sessions:** optional session-level history for statistics.
- **tags and book_tags:** optional flexible tagging.

Example future PostgreSQL books table

```
CREATE TABLE books ( id UUID PRIMARY KEY, user_id UUID NOT NULL, category_id UUID, title TEXT NOT NULL, author TEXT, description TEXT, file_path TEXT, cover_path TEXT, file_size BIGINT, mime_type TEXT, total_pages INTEGER DEFAULT 0, status TEXT DEFAULT 'not_started', created_at TIMESTAMPTZ DEFAULT NOW(), updated_at TIMESTAMPTZ DEFAULT NOW() );
```

For reading progress, enforce one record per user and book with a unique constraint on (user_id, book_id).

7. Suggested Project Structure

```
src/ app/ (dashboard)/ page.tsx library/ collections/ categories/ bookmarks/ statistics/ settings/
reader/[bookId]/ api/ books/ progress/ bookmarks/ uploads/ layout.tsx globals.css components/ ui/
layout/ dashboard/ library/ reader/ bookmarks/ forms/ features/ books/ reading/ bookmarks/
collections/ statistics/ lib/ db/ repositories/ storage/ validation/ utils/ hooks/ types/ config/
```

8. Development Roadmap

Phase 1 — Foundation and dashboard

- Inspect and run the starter project.
- Establish design tokens, typography, spacing, and responsive layout.
- Complete sidebar/navigation and dashboard states.
- Add project documentation and ensure lint/build checks pass.

Phase 2 — Local library and CRUD

- Create SQLite schema and migrations.
- Implement repository and local file-storage abstractions.
- Build upload validation and safe file handling.
- Implement create, edit, delete, search, filter, and sort.
- Add duplicate detection if practical.

Phase 3 — PDF reader

- Integrate PDF.js.
- Implement page navigation, zoom, fit modes, fullscreen, and keyboard controls.
- Add text search and outline support where feasible.
- Test reader on desktop and mobile browser sizes.

Phase 4 — Reading intelligence

- Track current page automatically and debounce saves.
- Resume at the last saved page.
- Add bookmarks, notes, and reading status.
- Introduce reading sessions and basic statistics if useful.

Phase 5 — Organization

- Complete categories and collections.
- Add tags and advanced filtering if needed.
- Improve metadata editing and book covers.

Phase 6 — Cloud migration

- Create Supabase project and PostgreSQL schema.
- Migrate local book records and map integer IDs to UUIDs.
- Configure Supabase Auth and private Storage buckets.
- Implement Row Level Security and per-user authorization.

- Migrate PDF files and verify record/file consistency.

Phase 7 — Multi-device and PWA

- Make the web app installable as a PWA.
- Test desktop and iPhone browser workflows.
- Add cloud progress synchronization and conflict handling.
- Add offline shell and carefully scoped caching.

Phase 8 — Production readiness

- Configure deployment and environment variables.
- Set upload limits, security controls, and monitoring.
- Implement backup and recovery procedures.
- Run acceptance, regression, and security tests.

9. Security and Reliability Requirements

- Validate file extension, MIME type, and file size; do not trust the filename alone.
- Generate safe server-side filenames and prevent path traversal.
- Use parameterized SQL queries and validate API inputs with Zod.
- Do not expose private file paths or unrestricted storage URLs.
- When cloud authentication is added, enforce per-user authorization and test Supabase Row Level Security.
- Use private storage buckets and authenticated access or short-lived signed URLs.
- Delete database metadata and the associated file consistently; handle partial failures.
- Back up the database and uploaded PDFs before production use.
- Test that one user cannot access another user's books or files.

10. Testing and Acceptance Criteria

Suggested testing layers

- **Unit tests:** validation, progress calculations, file naming, and utility functions.
- **Integration tests:** SQLite repositories and file-storage operations.
- **API tests:** upload, edit, delete, and progress endpoints.
- **UI tests:** navigation, search, filters, empty states, and error states.
- **End-to-end tests:** upload a PDF, open it, navigate, close it, reopen it, and resume.

MVP acceptance checklist

- A valid PDF can be uploaded and appears in the library.
- The user can search for and open the uploaded book.
- The reader can navigate between pages.
- Progress is saved and restored after closing and reopening the reader.
- Book metadata can be edited and the book can be deleted.
- Deleting a book also removes its stored PDF, with errors handled safely.

11. Codex and VS Code Working Method

Use Codex as the implementation assistant inside VS Code, while keeping each task narrow and reviewable. The local editor and terminal remain under the user's control.

1. Define one concrete task and its acceptance criteria.
2. Ask Codex to inspect the relevant files before changing anything.
3. Implement only the agreed scope; avoid broad rewrites.
4. Run lint, type-check, build, and relevant tests.
5. Review the diff and test the feature in the browser.
6. Fix regressions and commit a stable milestone to Git.

Recommended repository documentation

- **AGENTS.md:** coding conventions and instructions for Codex.
- **README.md:** setup, commands, and troubleshooting.
- **PROJECT_PLAN.md:** product scope and roadmap.
- **ARCHITECTURE.md:** layers, storage decisions, and data flow.
- **CHANGELOG.md:** completed changes by milestone.
- **docs/:** schema, API contracts, testing, and migration notes.

Codex guardrails

- Use strict TypeScript and small reusable components.
- Keep UI, business logic, database access, and file storage separated.
- Avoid adding dependencies without a clear reason.
- Do not introduce Supabase before the cloud-migration phase.
- Explain database migrations and destructive operations before running them.
- Never commit secrets or environment files containing credentials.

12. Deployment and Cost Strategy

Initial cost target: ■0. Develop and test locally with SQLite and local files. GitHub can be used for version control. Do not deploy the local-storage version as if it were a durable multi-device service.

Before public deployment, migrate the database and PDFs to persistent managed services, configure authentication and authorization, and verify upload limits and backup procedures. Hosting and storage providers may introduce costs as usage grows; confirm current pricing before choosing a production plan.

13. Future Enhancements

- AI-powered book questions and summaries.
- Text-to-speech and listening mode.
- EPUB support and broader ebook format support.
- Advanced reading analytics and personal reading goals.
- Import/export and backup tools.
- Multi-user or SaaS edition only if there is a validated product need.

14. Practical Starting Checklist

1. Extract the ReadVault starter ZIP and open the project folder in VS Code.
2. Install a supported Node.js LTS version and run npm install.
3. Run npm run dev and open http://localhost:3000.
4. Check the dashboard, navigation, upload flow, library, and reader placeholders.
5. Run the project's available lint/type-check/build commands and fix any issues.
6. Create a Git repository and commit the untouched starter baseline.
7. Implement Phase 1 acceptance criteria before moving to library CRUD.

Important implementation note

The starter project was generated as a starting point but was not verified by running npm install or a production build in the generation environment. Treat the first local run as a validation step. If installation or compilation fails, capture the complete terminal error and resolve it before building additional features.

15. Planning Estimate

A rough planning estimate for the full roadmap is 20–32 working days, depending on available time, debugging, polish, and the depth of reader functionality. This is an estimate, not a delivery commitment. The local MVP should be prioritized over optional enhancements.